Scanner Implementation

sábado, 17 de maio de 2025 09:52

- Starting reading a source file
  - May need to reate one as examle so i can read it.
    - Already create one for the functional tests
  - Start with a strategy pattern to get the content out of the source files
    - This one will only ready and return the raw data as a string
    - Utility class that defines a class SourceReaderAsString
    - Define an interface ISourceReader
      - This configuration allows us to swap implementaton latter on
    - **Strategy Pattern** is when you:
      - Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
      - You can swap file reading logic (e.g., for compressed files, cloud storage, etc.) without changing SourceScanner.
      - **Adheres to SOLID**

        This follows:
        - **S**: Single Responsibility — Reader and Scanner are separate.
        - **O**: Open/Closed — You can extend with new readers.
        - **D**: Dependency Inversion — Scanner depends on an interface, not a concrete class.

Scanner

raw source code

→ group into tokens

SWITCH

```java
package com.craftinginterpreters.lox;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.Charset;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.List;

public class Lox {
  public static void main(String[] args) throws IOException {
    if (args.length > 1) {
      System.out.println("Usage: jlox [script]");
      System.exit(64);
    } else if (args.length == 1) {
      runFile(args[0]);
    } else {
      runPrompt();
    }
  }
}
```

→ Receives an string as entry

→ I'll focus on this case
↳ given a source file, how can I run this!!

```java
private static void runFile(String path) throws IOException {
  byte[] bytes = Files.readAllBytes(Paths.get(path));
  run(new String(bytes, Charset.defaultCharset()));
}
```

→ Read everything as a string

→ Here is where everything starts

Both the prompt and the file runner are thin wrappers around this core function:

```java
private static void run(String source) {
  Scanner scanner = new Scanner(source);
  List<Token> tokens = scanner.scanTokens();

  // For now, just print the tokens.
  for (Token token : tokens) {
    System.out.println(token);
  }
}
```

→ Create a new scanner class

→ given that class we create a list of tokens

→ Scanner for Conditional Match
↳ Groups tokens per file

Token	Scanner
→ Represents a token ↳ maybe LF or something	→ scanTokens()

I'm creating a wrapper to handle multiple files entry

↳ Scanner For Condition Match class → Change when/for name!!!

→ Evaluate from content → vector < str file, content >

→ Group by tokens → Call Scanner for each file
↳ Separate threads!!

Great question! In C++, here are the **best practices for defining an enum** like TokenType, especially in a project involving an interface (IToken) and a concrete class (Token):

✅ Best Practices for enum TokenType

1. Prefer a Separate Header File (Best Practice)

Create TokenType.h to define the enum. This makes it easy to:
- Reuse in multiple files
- Keep responsibilities separated
- Avoid circular includes

```cpp
// TokenType.h
#pragma once
enum class TokenType {
  Identifier,
  Keyword,
  Literal,
  Operator,
  Comment,
  EndOfFile,
  // ...
};
```

You can then include it in both IToken and Token.

2. Use enum class
- Strongly typed
- Avoids polluting the global namespace
- Safer and more readable

✅ Good:

```cpp
enum class TokenType { Identifier, Keyword };
```

❌ Avoid:

```cpp
enum TokenType { Identifier, Keyword };
```

3. Don't Define Inside the Interface or Concrete Class
- TokenType is a domain-level concept, **not behavior-specific** to one class
- Keeping it in IToken or Token creates tight coupling
- If your architecture changes, you'll need to move it anyway

4. If You're Supporting Multiple Languages

Later, use separate enums or make TokenType extensible:

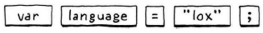
```cpp
// CppTokenType.h
enum class CppTokenType : int { Identifier, IncludeDirective, Namespace };
// JavaTokenType.h
enum class JavaTokenType : int { Identifier, Import, Package };
```

You can then abstract it using variants or adapters if needed.

Lexical Analysis

Scan through the list of chars & group them together into the smallest sequence that represents something

represent something. Each of these blobs of characters is called a **lexeme**. In that example line of code, the lexemes are:



- ~~Compare strings vs chars~~
- When recognizing a lexem we need to categorize the

Literal Values

- Numbers & strings
- make each char to identify it

Location Information

→ Useful for errors display
→ Line the token appears / col / length } → Create a class LF

Regexp

→ Core of a scanner is a loop starting from the source code to identify lexemes
↳ Creates a token at the end of the lexeme
↳ figure it out which lexem it makes

Lexical Grammar → The rules that determine how a particular language groups chars into lexemes
tools like Lex that you do this
↳ through regexps and they create scanners

The Scanner Class

→ string w/ source code } members variables !
→ List of tokens

```
    // You should store a list of std::shared_ptr<IToken> in your Scanner class,
not the concrete
    // Token objects.
    std::list<std::shared_ptr<IToken>> m_tokens;  // List of tokens generated
from the source code
```

Explanation on why to use IToken

✅ Summary Recommendation

Decision Point	Recommendation
Enum name	TokenType
Enum style	enum class
Location	Separate TokenType.h file
Used by	Both IToken and Token
Future extensibility	Split per language if needed

Let me know if you want a quick sample showing how to integrate TokenType into Token and IToken.

Great, you're thinking along the right architectural lines. Let's walk through this decision carefully and practically.

✅ Short Answer

You should **store a list of std::shared_ptr<IToken>** in your Scanner class, **not the concrete Token objects**.
That way:

- Your scanner works with **any future token implementation** (e.g., for other languages or test mocks).
- You **decouple** the scanner logic from the concrete Token class.
- You improve **testability and extensibility**.

💡 Why Prefer IToken over Token in the List?

✅ Benefits of using std::shared_ptr<IToken>:

1. **Abstraction**
   Scanner only needs to know the *interface* of a token, not the full implementation.
2. **Testability**
   You can easily inject MockToken objects using GoogleMock for unit tests.
3. **Extensibility**
   You might support multiple languages later (C++, Java...), with each having its own Token implementation — all inheriting from IToken.
4. **Looser Coupling**
   Keeps your Scanner class independent of concrete implementation details.

✍️ Refactored Example

```
#include <list>
#include <memory>
#include <string>
#include <vector>
#include "IScanner.h"
#include "IToken.h"  // <-- Interface only, not Token.h
class Scanner : public IScanner {
public:
    Scanner() = default;
std::vector<std::string> scanTokens(const std::string& rawSourceCode) const override;
private:
    std::string m_sourceCode;
    std::list<std::shared_ptr<IToken>> m_tokens;
};
```

🧠 When Would You Use the Concrete Class Token Instead?

Only if:

- You're 100% sure there will be only one implementation (i.e., Token) and no tests/mock needed

```
    std::string m_sourceCode;
    std::list<std::shared_ptr<IToken>> m_tokens;
};
```

When Would You Use the Concrete Class Token Instead?

Only if:

- You're **100% sure** there will be **only one implementation** (i.e., Token) and no tests/mock needed.
- Or it's **quick-and-dirty** prototyping.

But even then, **start with IToken** for long-term maintainability.

Summary Table

Option	Use Case	Pros	Cons
std::list<Token>	Small project, no mocking or variants	Simpler	Tightly coupled
std::list<std::shared_ptr<IToken>>	Best for long-term architecture / extensible	Decoupled, testable	Slightly more complex syntax

If you'd like, I can help you define IToken, Token, and a mock version for testing. Would you like a full code snippet?

maximal munch. When two
lexical grammar rules can both match a chunk of code that the scanner is
looking at, whichever one matches the most characters wins.
Maximal munch means we can't easily detect a reserved word until we've
reached the end of what might instead be an identifier

You are missing several TokenType enum values in both the toString function and the keywordsMap mapping. Here's a summary:

1. TokenType enum values missing in toString:

- PUBLIC
- PROTECTED
- THIS
- VOID

These are present in the enum but not handled in the toString switch (except for VOID, which is present, but THIS and PROTECTED are not).

2. TokenType enum values missing in keywordsMap:

- or (TokenType::OR)
- return (TokenType::RETURN)
- this (TokenType::THIS)
- true (TokenType::TRUE)
- while (TokenType::WHILE)
- void (TokenType::VOID)
- delete (TokenType::DELETE)
- new (TokenType::NEW)
- protected (TokenType::PROTECTED)
- public (TokenType::PUBLIC)

These are present in the enum and toString, but not in keywordsMap.

3. TokenType enum values in keywordsMap but not in toString:

- All keywords in keywordsMap are present in the enum, but some are missing in toString (see above).

Summary Table:

TokenType	In Enum	In toString	In keywordsMap
PUBLIC	✔	✗	✗
PROTECTED	✔	✗	✗
THIS	✔	✗	✗
VOID	✔	✔	✗
OR	✔	✔	✗
RETURN	✔	✔	✗
TRUE	✔	✔	✗
WHILE	✔	✔	✗
DELETE	✔	✔	✗
NEW	✔	✔	✗

Recommendation:

- Add missing cases for PUBLIC, PROTECTED, THIS in toString.
- Add missing keywords ("public", "protected", "this", "void", "or", "return", "true", "while", "delete", "new") to keywordsMap.

This will ensure all TokenType enum values are consistently handled in both toString and keywordsMap.